면접 준비 요약

> 목적: 면접 직전에 보는 압축본;

1. 30초 자기소개

저는 프론트엔드 중심으로 개발해왔고, 주로 `Vue` 와 `Svelte` 를 많이 사용했습니다. 이번 가계부 프로젝트는 제가 실제로 불편함을 느끼던 입력과 분류의 번거로움을 해결해보고 싶어서 시작했고, 동시에 `React` 와 `Next.js` 를 실전으로 익히기 위한 계기로 삼았습니다. 그래서 새로운 기술을 배우면서도 과한 스택으로 복잡도를 키우기보다, 실제로 끝까지 구현 가능한 구조를 우선했습니다.;

1-1. 엘리베이터 피치

> 가계부 입력이 귀찮아서 사용을 포기하는 사람들을 위해, 자연어 한 줄만 입력하면 AI가 카테고리, 금액, 날짜를 자동 분류해주는 가계부입니다. 은행 알림 메시지를 그대로 붙여넣거나 영수증 사진을 올려도 거래 초안으로 정리할 수 있습니다.;

---

2. 1분 프로젝트 설명

이 프로젝트는 로그인 이후 사용하는 앱형 가계부 서비스입니다. 제가 가계부를 쓸 때 가장 번거롭게 느낀 건 기록 자체보다 입력하고 분류하는 과정이어서, 자연어 입력이나 이미지 업로드만으로 AI가 거래를 파싱하고 정리해주는 흐름을 만들고 싶었습니다.;

동시에 저는 `React` , `Next.js` , 백엔드, DB를 실전으로 익히는 단계였기 때문에, 초반부터 너무 많은 스택을 얹어 개발 속도와 학습 효율을 같이 잃고 싶지는 않았습니다. 그래서 조회는 서버 컴포넌트, 저장은 `Server Actions` , 갱신은 `revalidatePath` 와 `router.refresh` 로 가져가는 `App Router` 중심 구조를 택했습니다.;

DB는 거래, 자산, 예산처럼 관계와 집계가 중요한 도메인이라 `PostgreSQL + Supabase` 를 선택했고, 상태관리도 현재 규모에서는 글로벌 상태보다 로컬 UI 상태가 많아서 `Zustand` 와 `React Query` 는 의도적으로 넣지 않았습니다. 그 선택 덕분에 핵심 기능과 데이터 정합성에는 집중할 수 있었지만, 웹접근성, 시맨틱, SEO, 클라이언트 캐시 UX는 다음 단계 보완 과제로 남았습니다.;

---

3. 핵심 한 줄 답변

왜 만들었나

> 가계부의 기록보다 입력과 분류 과정이 번거롭다고 느껴서, AI가 자연어와 이미지를 파싱해주는 가계부를 만들고 싶었습니다.;

왜 App Router + Server Actions인가

> 로그인 기반 앱형 서비스에서 서버 중심 데이터 흐름과 정합성을 단순하게 가져가기 위해서였습니다.;

왜 Supabase + PostgreSQL인가

> 거래, 카테고리, 자산, 예산처럼 관계와 집계가 중요한 도메인이라 관계형 DB가 맞았고, 입문자 관점에서도 자료가 많고 검증된 조합이 필요했습니다.;

왜 Firebase 같은 NoSQL이 아니었나

> 이 프로젝트는 실시간성보다 거래, 카테고리, 계좌 간 참조 무결성과 집계 쿼리가 더 중요했습니다. 그래서 스키마리스의 유연함보다 관계형 모델의 안정성이 더 적합하다고 판단했습니다.;

왜 Drizzle인가

> TypeScript 코드 자체가 스키마와 타입 역할을 같이 하기 때문에, 별도 generate 단계 없이 일관성을 유지하기 좋았습니다.;

왜 Better Auth인가

> 기존 DB와 인증 레이어를 더 자연스럽게 붙일 수 있었고, 가입 직후 기본 카테고리 시딩이나 세션 저장 전 최소화 처리 같은 DB 훅을 인증 흐름에 직접 연결하기 쉬웠습니다.;

왜 Zustand를 안 썼나

> 현재 프로젝트는 전역 상태가 크지 않고 대부분 로컬 UI 상태였기 때문에, 상태관리 계층을 추가하는 것보다 단순하게 유지하는 편이 더 적절하다고 봤습니다.;

왜 React Query를 안 썼나

> 조회는 서버 컴포넌트, 수정은 Server Actions, 갱신은 revalidatePath와 router.refresh로 이미 정리돼 있어서 현재 규모에서는 캐시 계층 이중화가 더 부담이라고 판단했습니다.;

왜 접근성이 약한가

> 몰라서 놓친 게 아니라, 로그인 기반 앱형 서비스라 초기 우선순위를 인증, AI 파싱, 데이터 정합성, 모바일 UX에 뒀기 때문입니다. 다만 에이전시 기준으로는 분명 보완이 필요한 상태입니다.;

왜 Capacitor를 붙였나

> 가계부는 모바일 사용 빈도가 높기 때문에, 하나의 웹 코드베이스로 앱 확장성을 검증하는 것이 초기 단계에서 현실적인 선택이라고 봤습니다.;

AI가 많이 썼는데 본인 기여는 무엇인가

> AI가 구현 속도를 높이긴 했지만, 문제 정의, 구조 선택, 기술 채택, 검증 기준, 최종 책임은 제가 가졌습니다.;

AI 파이프라인은 어떻게 흘렀나

> 입력 검증, OOD 필터, 은행 메시지 전처리, provider 선택, DB 조회, 프롬프트 조립, LLM 호출, 응답 검증, 확인 시트, 확정 저장 순서로 흘렀고, 핵심은 자동 저장보다 사용자 확인 가능한 구조를 유지하는 것이었습니다.;

보안 입력 검증은 어떻게 했나

> 공개 입력은 길이 제한만 둔 게 아니라, `base64` 위장 텍스트 차단, 이미지 MIME/크기/실제 `base64` 검증, Origin 검증, rate limit까지 함께 적용했습니다.;

---

4. 약점과 보완 계획

현재 약점

- 웹표준, 시맨틱, 접근성, SEO 완성도 부족;
- `React Query` 미도입으로 optimistic update, background refetch UX 약함;
- `Zustand` 미도입으로 향후 전역 상태가 커지면 구조 재검토 필요;
- `Capacitor` 기반이라 웹과 앱 사이의 긴장이 존재함;

다시 한다면

- 시맨틱 구조와 접근성 기준을 더 초기에 설계에 포함;
- 확대 허용과 포커스 관리 보강;
- 메타데이터와 공개 페이지 구조 정리;
- 규모가 커지면 `React Query` 와 `Zustand` 부분 도입 검토;

---

5. 절대 피해야 할 말

- `시간이 없어서 못 했습니다.`
- `앱이라 접근성은 덜 중요했습니다.`
- `거의 AI가 다 했습니다.`
- `React Query는 필요 없다고 생각합니다.`
- `SEO는 로그인 서비스라 필요 없습니다.`

---

6. 대신 이렇게 말하기

- `우선순위상 뒤로 둔 것이고, 다시 한다면 가장 먼저 보완할 영역입니다.`
- `현재 구조에서 복잡도와 이득을 비교해 의도적으로 넣지 않았습니다.`
- `AI가 구현 속도를 높였지만, 설계와 검증 책임은 제가 가졌습니다.`
- `장점뿐 아니라 트레이드오프도 함께 설명할 수 있어야 한다고 생각합니다.`

---

7. 마지막 체크

면접 직전에는 아래만 다시 본다.;

- 프로젝트 한 줄 정의;
- 왜 만들었는가;
- 왜 이 스택을 택했는가;
- 왜 안 넣은 기술이 있는가;
- 현재 약점은 무엇인가;
- 다시 한다면 무엇을 먼저 바꿀 것인가;
- 공개 입력 보안과 `LLM` 응답 검증을 코드 수준으로 설명할 수 있는가;